# How to Create a Customizable Relative Rotat Graph (RRG) Using Python

Empowering DIY Investors with Flexible Market Analysis Tools

FACTORLAB
JUL 28, 2024

8 2

# How to Create a Customizable Relativ Rotation Graph (RRG) Using Python

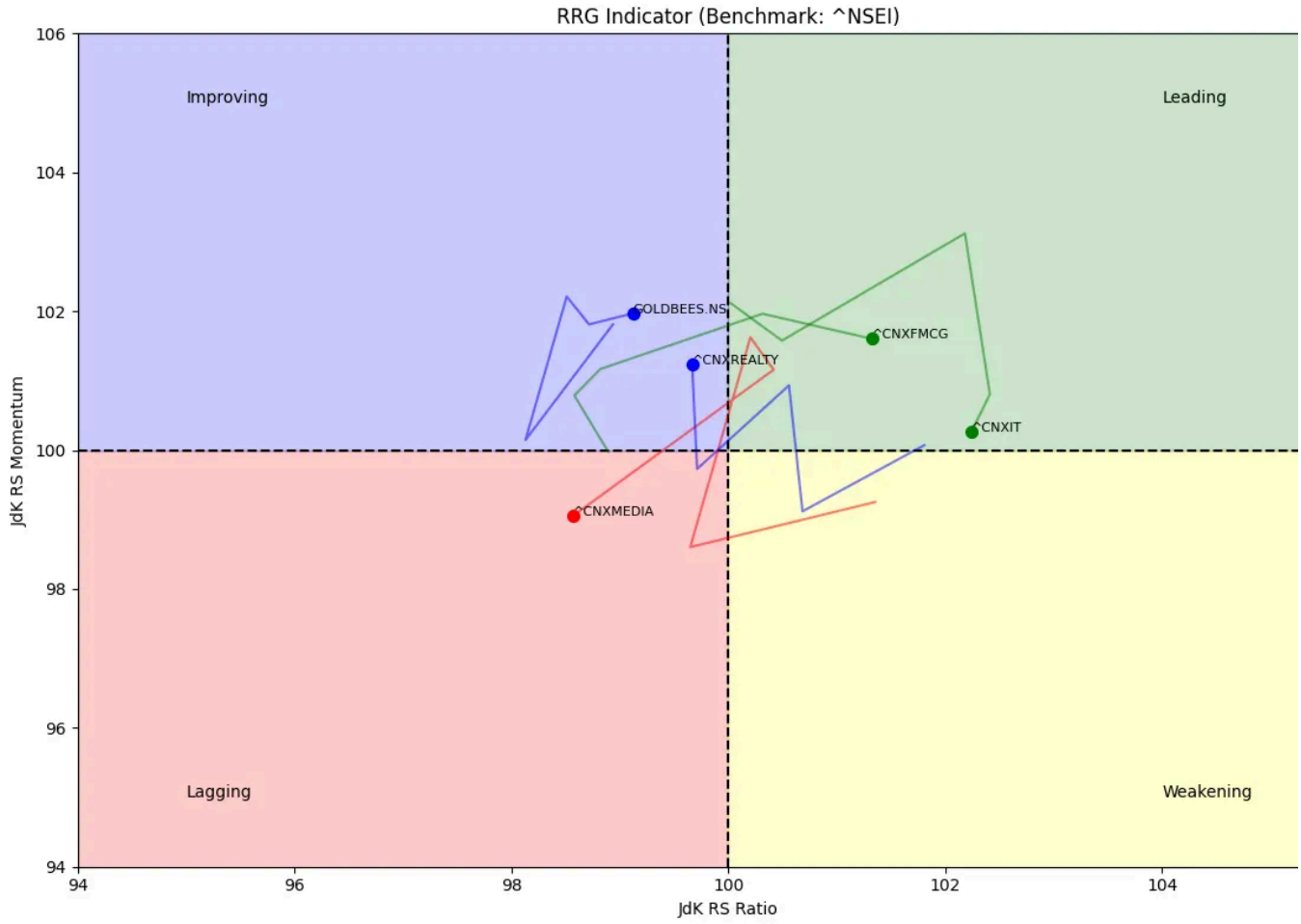
## Empowering DIY Investors with Flexible Marke Analysis Tools

### Introduction

Relative Rotation Graphs (RRGs) are powerful tools for visualizing asset perform and sector rotation. While most available RRG tools are either expensive or lack customization options, today we'll introduce a unique Python-based RRG plotter puts the power of customization in the hands of DIY investors.

### What is a Relative Rotation Graph (RRG)?

An RRG chart plots the relative performance of multiple securities against a com benchmark. It highlights their relative strength (RS) and momentum (RS-Momen providing insights into performance trends. Each point on the graph represents security's position at a specific time, with trajectories showing relative movemer




# Why Use This Customizable RRG Chart?

1. **User Input**: Choose your own benchmark and tickers
2. **Flexibility**: Adjustable time period and calculation window
3. **Visualization**: Both static and animated plots
4. **Cost-effective**: Free to use and modify
5. **Educational**: Understand underlying calculations

# Step-by-Step Guide to Creating a Customizable RRG Ch

## 1. Setup and Install Dependencies

```
pip install pandas numpy yfinance matplotlib
```

## 2. Import Libraries

```
import pandas as pd
import numpy as np
import yfinance as yf
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from IPython.display import HTML
```

## 3. Define Helper Functions

```
def get_status(x, y):
    if x < 100 and y < 100:
        return 'lagging'
    elif x > 100 and y > 100:
        return 'leading'
    elif x < 100 and y > 100:
        return 'improving'
    elif x > 100 and y < 100:
        return 'weakening'

def get_color(x, y):
    if get_status(x, y) == 'lagging':
        return 'red'
    elif get_status(x, y) == 'leading':
        return 'green'
    elif get_status(x, y) == 'improving':
        return 'blue'
    elif get_status(x, y) == 'weakening':
        return 'yellow'
```

## 4. Create the RRGPlotter Class

```
class RRGPlotter:
    def __init__(self, benchmark, tickers, period='2y', window=12, tail=5):
        self.benchmark = benchmark
        self.tickers = tickers[:10]  # Limit to max 10 tickers
        self.period = period
        self.window = window
        self.tail = tail
        self.load_data()
        self.calculate_rrg()
```

```
        self.create_plot()

    def load_data(self):
        self.tickers_data = yf.download(self.tickers, period=self.period, interval="1wk")['A
Close']
        self.benchmark_data = yf.download(self.benchmark, period=self.period, interval="1wk"
['Adj Close']

    def calculate_rrg(self):
        # RRG calculations (RS, RS-Ratio, RS-Momentum)
        for ticker in self.tickers:
            rs = 100 * (self.tickers_data[ticker] / self.benchmark_data)
            rsr = (100 + (rs - rs.rolling(window=self.window).mean()) /
rs.rolling(window=self.window).std(ddof=0)).dropna()

            if not rsr.empty:
                rsr_roc = 100 * ((rsr / rsr.shift(1)) - 1)
                rsm = (101 + ((rsr_roc - rsr_roc.rolling(window=self.window).mean()) /
rsr_roc.rolling(window=self.window).std(ddof=0))).dropna()

                # Append calculated values to respective lists
                ...

    def create_plot(self):
        # Create static RRG plot
        ...

    def animate(self, num_frames=60):
        # Create animated RRG plot
        ...
```

## 5. Get User Input

```
def get_user_input():
    benchmark = input("Enter the benchmark index ticker (e.g., ^NSEI for Nifty 50): ")
    tickers = input("Enter up to 10 tickers separated by commas (e.g.,
^CNXFMCG,^CNXIT,GOLDBEES.NS): ").split(',')
    tickers = [ticker.strip() for ticker in tickers[:10]]  # Limit to max 10 tickers
    period = input("Enter the period (e.g., 1y, 2y, 5y): ")
    window = int(input("Enter the window size for calculations (e.g., 12): "))
    tail = int(input("Enter the tail length for plotting (e.g., 5): "))
```

```
    return benchmark, tickers, period, window, tail
```

## 6. Main Function

```
def main():
    benchmark, tickers, period, window, tail = get_user_input()
    rrg_plotter = RRGPlotter(benchmark, tickers, period, window, tail)

    # Display static plot
    rrg_plotter.create_plot()

    # Display animated plot
    HTML(rrg_plotter.animate().data)

    # Print table with ticker information
    print(f"{'Ticker':<15}{'Name':<30}{'Price':<10}{'Change (%)':<10}{'Status':<10}")
    print("-" * 75)
    for i, ticker in enumerate(tickers):
        info = yf.Ticker(ticker).info
        name = info.get('longName', 'N/A')[:30]
        price = round(rrg_plotter.tickers_data[ticker].iloc[-1], 2)
        chg = round((price - rrg_plotter.tickers_data[ticker].iloc[0]) /
rrg_plotter.tickers_data[ticker].iloc[0] * 100, 2)
        status = get_status(rrg_plotter.rsr_tickers[i].iloc[-1],
rrg_plotter.rsm_tickers[i].iloc[-1])
        print(f"{ticker:<15}{name:<30}{price:<10.2f}{chg:<10.2f}{status:<10}")

if __name__ == "__main__":
    main()
```

---

# Interpreting the RRG Chart

- **Quadrant Analysis**: Assets in the top-right quadrant are outperforming the benchmark with increasing momentum. Those in the bottom-left quadrant a underperforming with decreasing momentum.

- **Latest Data Points**: The scatter points represent the most recent data point each asset, indicating their current position in terms of relative strength and momentum.
- **Tail**: The lines show the recent trajectory of each asset, helping to identify t

## Tips for Improvement

- **Extend Analysis**: Modify the code to include more asset classes or different frames.
- **Automate Updates**: Set up a script to run this analysis periodically for conti monitoring.
- **Add More Visualizations**: Incorporate additional charts or metrics to compl the RRG analysis.

## Conclusion

This customizable RRG chart using Python offers a unique combination of flexib visualization, and educational value. By allowing users to input their preferences empowers DIY investors to perform sophisticated technical analysis tailored to t specific needs and interests. Whether you're tracking sector rotation, comparing performance, or analyzing ETFs, this tool provides valuable insights to inform yo investment decisions.

Feel free to download the code from our GitHub and try it with your data. Happy analyzing!

---

## QuantX-Builder for Mastering Momentum Investing. 4 week comprehensive cohort learning programme.

## https://sekharsudhamsh.graphy.com/s/store

Feel free to leave your comments, questions, and thoughts below. Happy coding investing!

# Important Links

- Youtube: https://www.youtube.com/@MomentumLab_IN
- Twitter: https://x.com/MomentumLab_IN

**Disclaimer:** We are not SEBI registered advisors. Any content shared on or throu our digital media channels is for information and education purposes only and sl not be treated as investment or trading advice.

**Subscribe to factorlab Research Notes**

Launched 2 years ago

Systematic investing insights from FactorLab. We share quantitative research, factor strategies, and p optimization techniques that actually work in Indian markets. Real data, real strategies, real resul

Type your email... Subscribe

By subscribing, you agree to Substack's Terms of Use, and acknowledge its Information Collection Notice and Privacy Policy.



8 Likes

## Discussion about this post

Comments Restacks



Write a comment...



**Srijita Roy** Apr 25, 2025

I am getting and adjacent close error. Can you share a modified code?

KeyError Traceback (most recent call last)

File ~/miniconda3/lib/python3.12/site-packages/pandas/core/indexes/base.py:3805, in Index.get_loc(self, key)

3804 try:

-> 3805 return self._engine.get_loc(casted_key)

3806 except KeyError as err:

File index.pyx:167, in pandas._libs.index.IndexEngine.get_loc()

File index.pyx:196, in pandas._libs.index.IndexEngine.get_loc()

File pandas/_libs/hashtable_class_helper.pxi:7081, in pandas._libs.hashtable.PyObjectHashTable.get_item()

File pandas/_libs/hashtable_class_helper.pxi:7089, in pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'Adj Close'

The above exception was the direct cause of the following exception:

KeyError Traceback (most recent call last)

Cell In[23], line 24, in RRGPlotter.load_data(self)

23 try:

---> 24 self.tickers_data = raw_data.xs('Adj Close', axis=1, level=0)

See more

LIKE REPLY

**1 reply by factorlab**

**1 more comment...**

© 2026 MomentumLAB · Privacy · Terms · Collection notice
Substack is the home for great culture